

系統架構課 · 分享會

Plannotator × Obsidian

把一次 AI 討論，變成可審查、可累積、可分享的工作成果

數位創新部 / 系統架構課 · 2026-08

你有沒有遇過這些情況？

這些是我個人在使用 AI 時遇到的困境，想跟大家確認：這些問題是不是也是你們的痛？

● AI 對話紀錄 · 真實情境



我

幫我寫一份產品企劃書，要包含市場分析和執行計畫



AI

好的，以下是企劃書草案：

一、市場概況

根據最新研究，目標市場規模預計成長 23%...

二、執行計畫

Phase 1：需求調研（2 週）

Phase 2：原型開發（4 週）

（全文 2000 字...）



我

這不是我要的 😞 市場分析太籠統，執行計畫也沒有考慮到我們的資源限制...

痛點 一次講不清楚，來回修改耗費時間

PAIN POINT 01



溝通不順

有時候和 AI 討論東西，它出來的規劃有多處我不是很認同，或者我覺得還可以再調整。

但往往很難一次性地全部表達，讓 AI 一次統整性地去思考。



表達瓶頸：

難以精確傳達多處結構性修改意見



修補泥沼：

缺乏收斂與審查機制，來回修補耗時

PAIN POINT 02



資訊分散

過往常常讀到很多資料，它們之間都是有關聯的，但資料總是被散落各處。

這些資料似乎是跟上次看過的什麼東西有關？有什麼辦法可以幫我**把資料之間的關聯整理出來**？



脈絡斷層：

關鍵對話細節散落，難以檢索與複用



難以累積：

經驗未能結構化沉澱至知識庫



如果你也有同樣的煩惱，那 **Plannotator** & **Obsidian** 或許能幫你？

什麼是 Loop Engineering ?

最近很紅的詞，用白話解釋就是：在讓 AI 做事之前，先讓它說清楚「它打算怎麼做」。

Loop Engineering 的核心：

AI 不是一次回答完，而是透過**問題** → **工具** → **證據** → **回饋**，逐步收斂到我們要的結果。

過去的做法：Single-shot Prompting

一次把需求講清楚，期待 AI 一次到位生成完整結果。

問題：

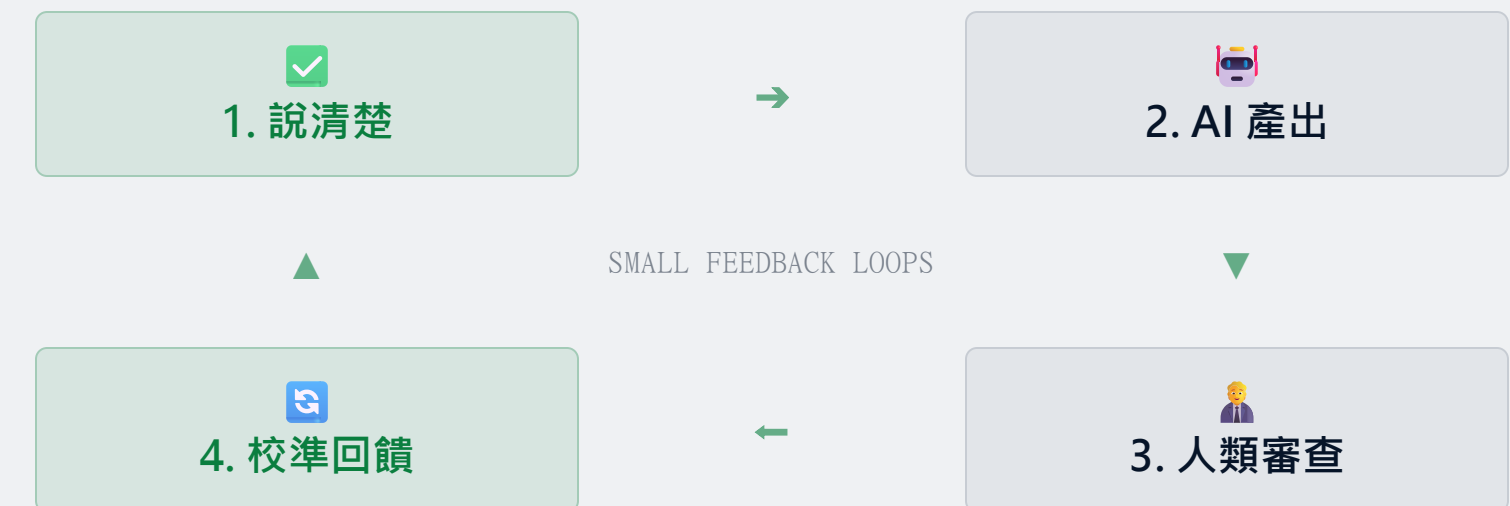
複雜的架構或問題的規劃不可能一蹴可幾，結果常有落差，只能整段打掉重講。

10+

次來回溝通才能收斂

現在的做法：Small Feedback Loops

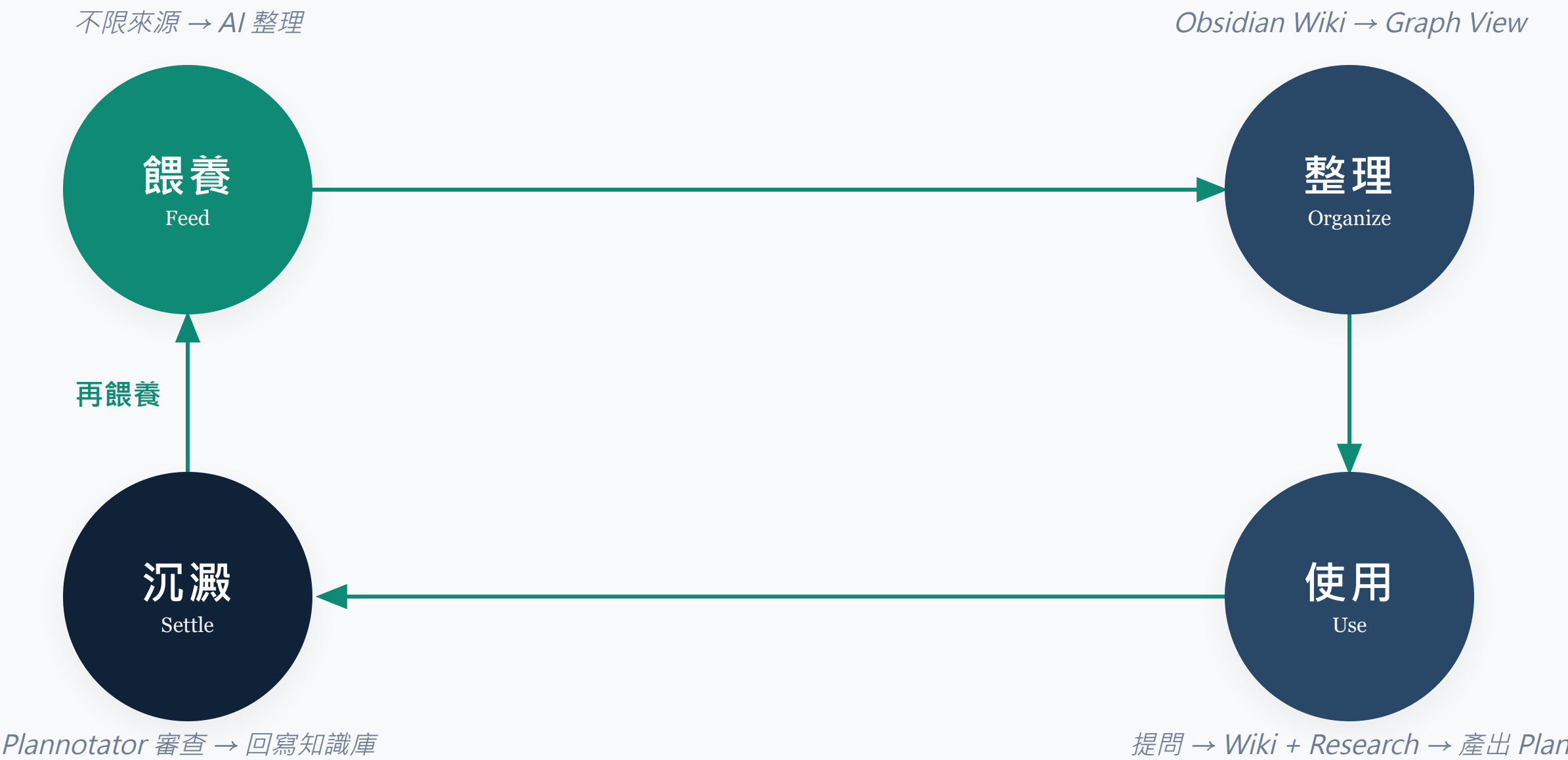
打造快速、微小的閉環：說清楚 → 產出 → 審查 → 校準，反覆收斂。



一句話重點： 用短迭代讓 AI 逐步達成可驗證的計畫。

創造知識的閉環迴流

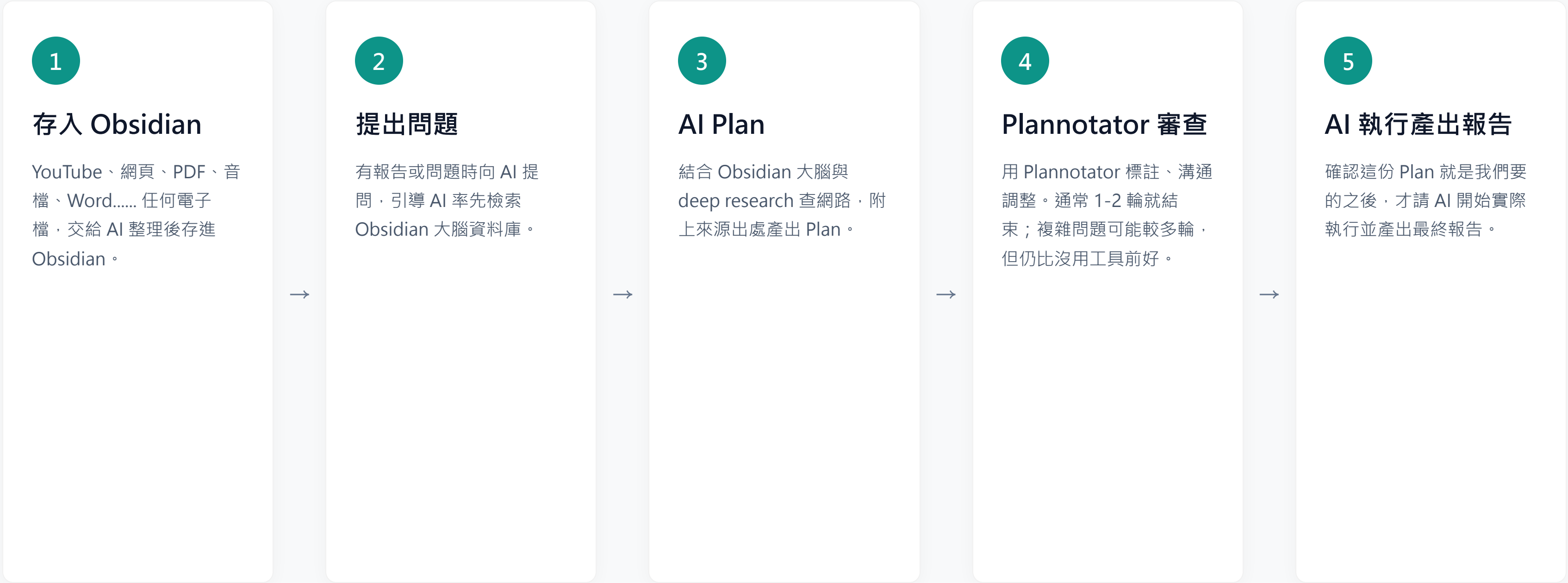
我們希望可以不限資訊的來源，透過 AI 整理後沉澱到知識庫；當有問題時，再從知識庫出發，結合 AI 研究，產出成果。



重點：這是一個閉環迴流，餵養 → 整理 → 使用 → 沉澱 → 再餵養，讓知識越用越聰明。

那廢話不多說，我們先示範案例

我們挑一個實際的情境，走一遍從「存入 Obsidian」到「提出問題」到「AI Plan」到「Plannotator 審查」到「AI 執行產出報告」的完整流程。



真實情境示範，帶你親眼見證知識迴流的端到端閉環流轉。

實機演示

現在，讓我們實際操作

從「存入 Obsidian」到「提出問題」到「AI 產出 Plan」到「Plannotator 審查」的完整流程。

 現場演示

安裝與落地

三步驟完成環境部署：自動安裝 → 手動微調 → 合規驗證，讓團隊快速上手。

推薦方法 讓 AI 自己動手裝

開啟 terminal 輸入以下指令：

```
npx skills@latest add  
https://github.com/EVACopilot/skills-itd-aas
```

選擇安裝 Skill - plannotator-copilot-setup 和 wiki-init

然後在 Copilot 對話框（不是終端機）輸入：

```
/plannotator-copilot-setup
```

Plugin 加入 backnotprop/plannotator 後，安裝 `plannotator-copilot@plannotator`

然後在 Copilot 對話框（不是終端機）輸入：

```
/wiki-init
```

AI 會自動初始化 Obsidian 知識庫結構。

由 skills-itd-aas 統一治理與分發，跟企業 Agent Skill 共享機制是同一套。

接下來 3 件事仍要手動做

- 1 安裝 Copilot 外掛（貼上兩行指令，或 GUI 點 Install）
- 2 完整重新啟動 GitHub Copilot App
- 3 Shift+Tab 進入 Plan Mode 驗證

隱私與資安

- ✓ 核心 Plan / Code / Annotation 全程在 localhost 執行
- ✓ 公司政策只核准 Copilot 時，不要額外安裝其他 AI CLI
- ✓  敏感資料未經脫敏不得輸入公有雲 AI 模型

一句話重點：輕量化配置、合規守護，3 步驟即可完成團隊級別的環境部署。

回顧實作流程

每個階段的明細步驟，串成一個完整的知識迴流循環。



Obsidian：第二大腦與 Knowledge Graph

源自 Andrej Karpathy 的 LLM Wiki 想法，以及 Google 的 OKF（Open Knowledge Framework）精神。

資料流



不需要向量資料庫：個人規模下，*wiki + index* 就夠用。Markdown 是可長期保存、可被 AI 讀寫的知識格式。

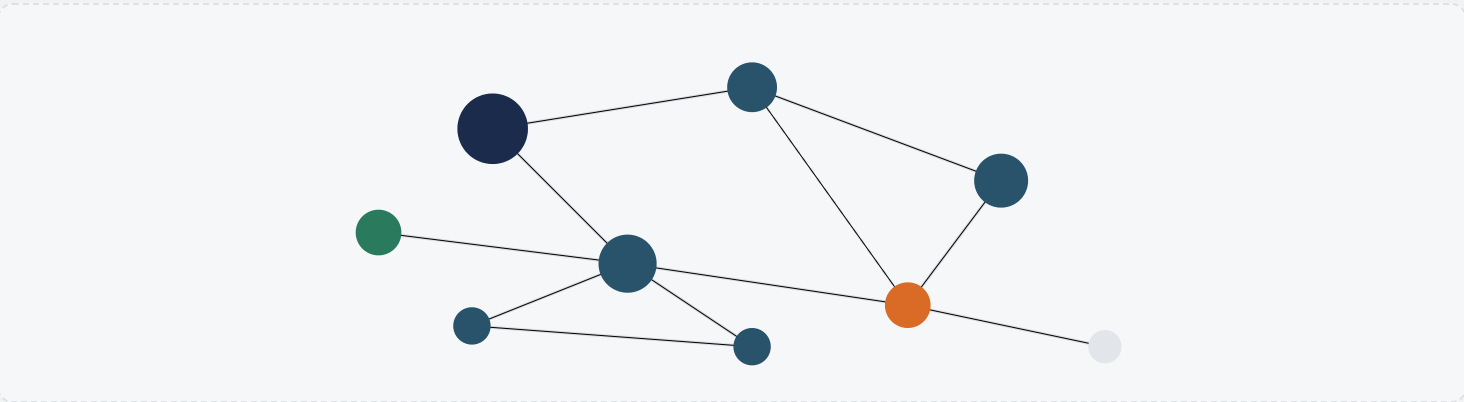
核心 SKILL

wiki-ingest 將 raw/ 資料自動攝取、轉換並歸檔到 wiki/

wiki-query 高精度關聯查詢，快速找到相關知識節點

Graph View 的四個價值

不是炫技的圖，而是系統性治理判斷工具：



品質控管 SKILL

wiki-lint 語意連結校驗，自動偵測斷裂連結與孤兒頁



發現關聯

節點間雙向連結，顯示跨領域概念如何互相支撐



知識缺口

揭露論述中應該存在、卻缺少的關鍵連結



孤兒頁

識別沒有任何關聯連結的獨立筆記

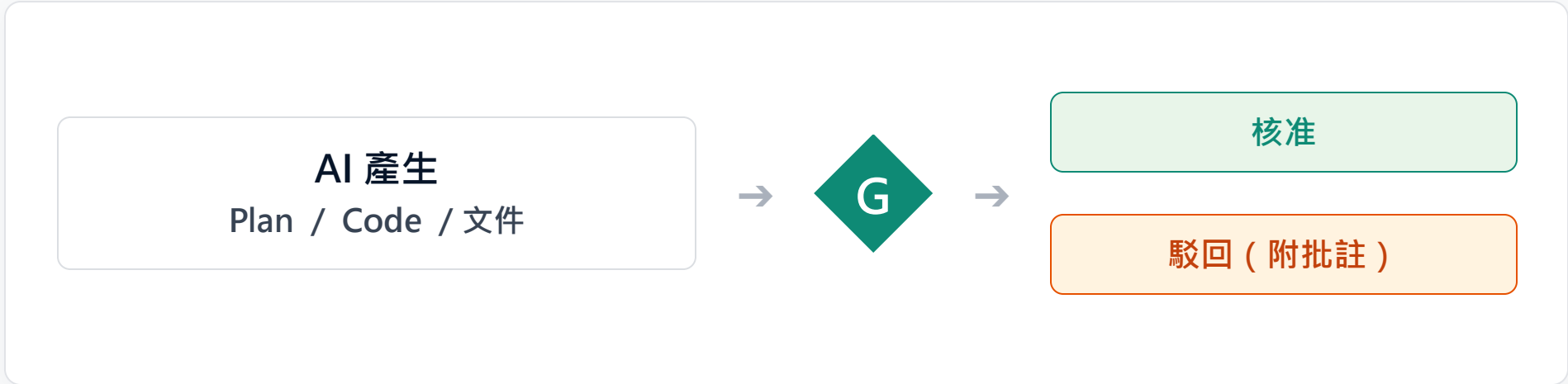


決策樞紐

掌握連結數量最多、影響力最大的核心節點

審查閘門：Plannotator

循環中負責「審查」與「控制」的工具——不只是 coding tool，任何 AI 產出都能納入審查。



任何符合格式的檔案都能進這個循環：計畫、程式碼、Markdown、HTML、URL。

常用指令

Plan Review

Shift+Tab 進入 Plan Mode

Code Review

/plannotator-review

Markdown / HTML 標註

/plannotator-annotate <路徑>

標註最後一則回覆

/plannotator-last

重點：可以單獨使用

不一定要搭配 Obsidian。任何跟 AI 討論時，想一次性表達所有反饋，都很適合用 Plannotator——像系統開發中的 PR 審查，在發佈前先做 Inspection。

個人經驗

10+ → 1-2

複雜的問題可能仍需較多輪，但比沒用工具前好很多

一句話重點：AI 說「我打算這樣做」時，讓你在執行前先審查它的計畫。

整合實踐：審查一次，沉澱永久

Plannotator 內建 Obsidian Integration，這是 Review Gate 與知識庫之間真正的橋樑。

Settings - Obsidian

Obsidian Integration **ON**

Vault

C:\CheerSpace\90_Wiki

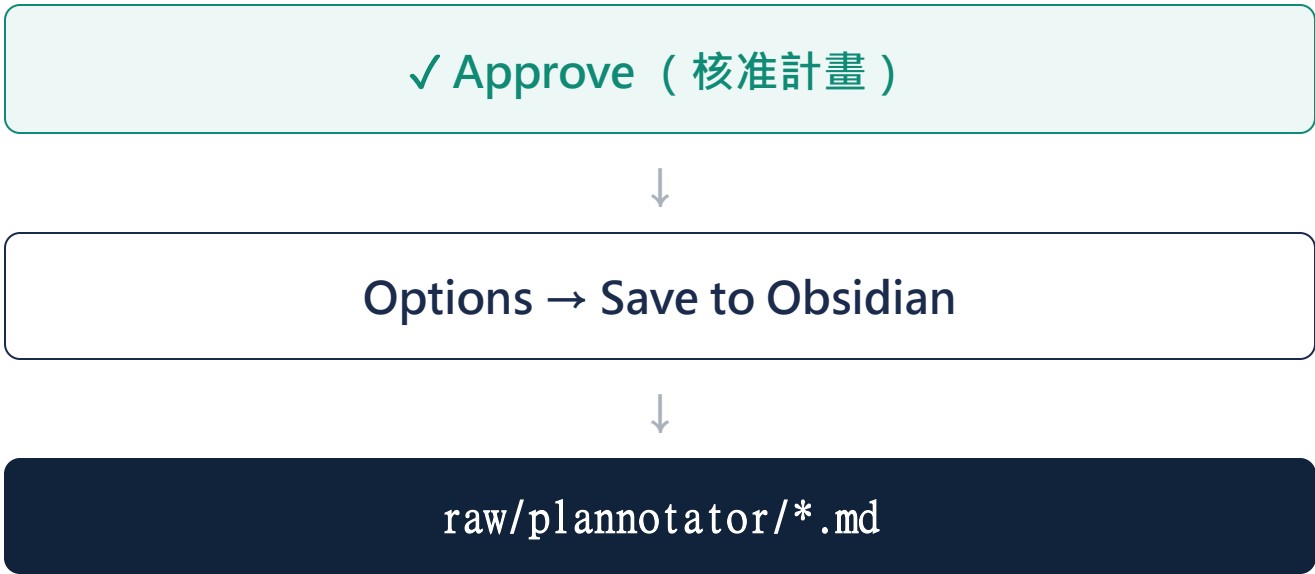
Folder

raw/plannotator

Frontmatter (自動產生)

```
created: 2026-07-23T08:14:00Z
source: plannotator
tags: [plan, ...]
```

實際操作流程

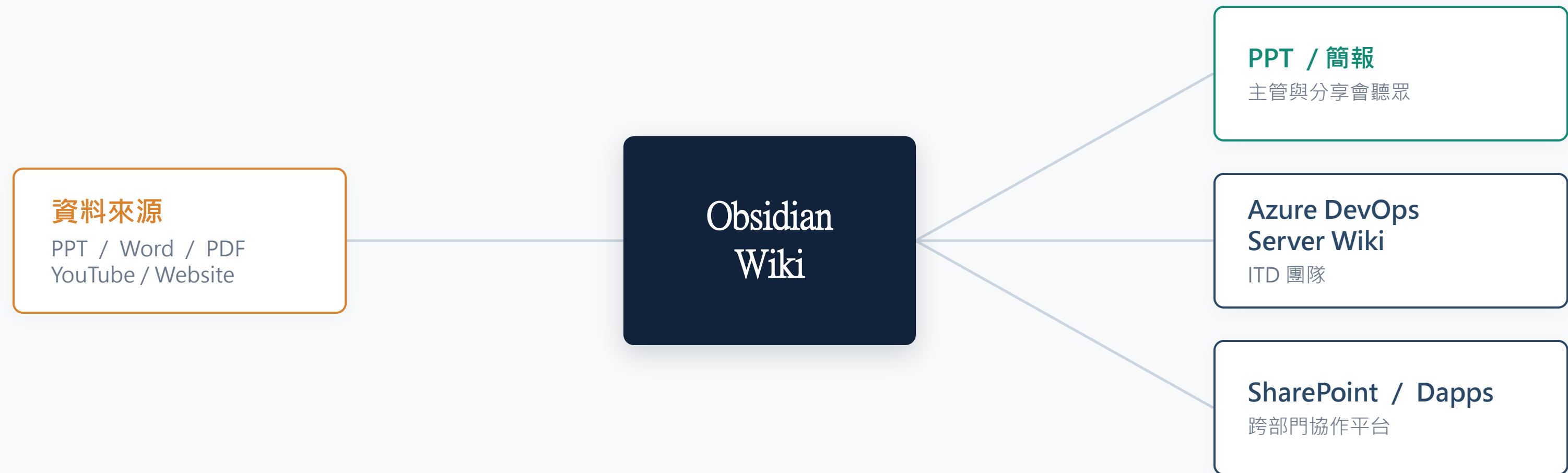


帶著 frontmatter 直接進 vault，0 手動搬運。

一句話重點：審查一次，沉澱永久——無縫將純淨知識寫入第二大腦。兩套工具不用擺在一起，但串起來就是完整迴流。

從同一份知識，輸出多種成果

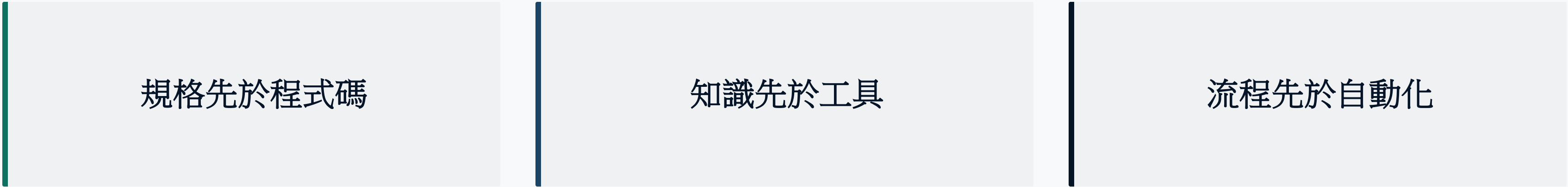
同一份結構化知識 (Single Source of Truth)，可以彈性轉譯成不同受眾需要的樣子。



一句話重點: 一次撰寫，多處輸出——讓同一份 Wiki 資產彈性轉譯為簡報、文件與 SOP。

這套流程能帶走什麼

用熟了這套方法之後，它可以這樣延伸——不用一次到位，慢慢養成。



明天開始的一個行動： 下一次和 AI 討論前, 先問「你打算怎麼回答？」——讓計畫先於執行。

感謝聆聽！

Q & A

歡迎提問與交流

這是我個人摸索出來的工作方式。
工具可以用別的，流程可以不一樣。
重要的是：找到適合你自己的方式。

資料來源

1	Plannotator 官方文件與原始碼	<i>plannotator.ai/docs • github.com/backnotprop/plannotator</i>
2	Obsidian 官方網站與知識管理理念	<i>obsidian.md</i>
3	Andrej Karpathy：LLM Wiki 概念（詳見下一頁）	見附錄
4	Google OKF（Open Knowledge Framework）指南（詳見下一頁）	見附錄
5	Human-in-the-Loop（HITL）系統設計規範與實踐	<i>en.wikipedia.org/wiki/Human-in-the-loop</i>
6	Prompt Engineering vs Loop Engineering 技術變革討論	近期技術社群討論

靈感來源： Andrej Karpathy 的 LLM Wiki

2026 年 4 月，Karpathy (OpenAI 共同創辦人、前 Tesla AI 總監) 發佈了一篇 gist，提出用 LLM 取代傳統 RAG 的記憶方法。

🔗 核心洞見

RAG 像是每次都重新「解讀」原始文件——每個問題都要重新拆解、重新拼湊片段，知識不會累積。

LLM Wiki 把 LLM 當成「知識編譯器」：讀一次原始資料，就把它「編譯」成結構化的 Wiki 頁面。往後的提問，直接命中 Wiki 層，得到的是連貫、有引用的答案，而不是拼湊的片段。

🏗️ 三層架構

- raw/** 原始來源，只增不改 (append-only)。錯了就補一份修正文件，不直接改原檔——保留完整稽核軌跡。
- wiki/** LLM 專屬的寫入層。每個檔案是一個主題的 Markdown 頁面：摘要、關鍵事實、相關頁面、回指原始來源的引用。
- schema** 一份設定文件 (像 AGENTS.md)，告訴 LLM Wiki 怎麼組織、命名慣例、更新規則——讓 LLM 是「有紀律的 Wiki 維護者」，不是隨性亂寫的聊天機器人。

跟我們的關聯：
AGENTS.md 裡的 raw/ 與 wiki/ (concepts / entities / sources / decisions / discussions) 分層，正是套用這套三層架構。
Karpathy 說：「 Obsidian 是 IDE，LLM 是工程師，Wiki 是程式碼庫。」

靈感來源： Google OKF

2026 年 6 月 12 日，Google Cloud 發布 Open Knowledge Framework (OKF) v0.1——一套開放、廠商中立的標準。

🔗 OKF 想解決什麼問題？

過去幾年，Retrieval-Augmented Generation (RAG) 是讓 AI 存取外部知識最主流的方法。但隨著 AI 應用越來越進階，光靠「檢索」已經不夠——AI 系統需要更好的方式去**理解、組織、並推理知識**，而不只是抓取相關片段。

OKF 正是為了 formalize 「企業內部資料與情境該如何被封裝，才能有效交給 AI Agent 使用」而提出的標準——它不是要取代 RAG，而是解決 RAG 沒解決的問題：知識的結構化與可組織性。

↔ 跟我們的關聯

我們的 Obsidian Wiki 分層設計 (concepts / entities / sources / decisions / discussions)，呼應了 OKF 「將企業知識結構化封裝，而非只靠向量檢索」的精神——這也是為什麼 Graph View 的知識缺口與決策樞紐分析，對架構治理特別有意義。

備註： OKF 是 2026 年 6 月才發布的早期 (v0.1) 標準，內容仍在演進中，此處僅作方向性參考。